

Read-Write Conflict & Unrepeatable Read

Detailed analysis of concurrent operation conflicts, sequence flows, and lost updates

Exam-Focused Definition

A **Read-Write Conflict** occurs in a concurrent/parallel schedule when a transaction reads a data item and another transaction modifies (writes) that same data item. This can cause the first transaction to observe different values when reading the same data again (known as an **Unrepeatable Read**).

1. OPERATION COMBINATIONS MATRIX

When two transactions operate concurrently on the same data item A, four possible combinations arise:

Case	Operations	Conflict Status	Associated Problem
Case 1	Read → Read	✗ No Conflict	None (Shared access allowed)
Case 2	Read → Write	⚠ Conflict	Unrepeatable Read
Case 3	Write → Read	⚠ Conflict	Dirty Read (Write-Read Conflict)
Case 4	Write → Write	⚠ Conflict	Lost Update

2. READ → READ COMBINATION (NO CONFLICT)

When two or more transactions perform a **READ** on the same data item, no data state changes occur in the database.

Example — Railway Seat Reservation

Initial Database State: Available Seats (A) = 2

- User 1 (T1): Executes **READ(A)** → Receives 2
- User 2 (T2): Executes **READ(A)** → Receives 2

Result: Both transactions read consistent data. Concurrent read operations cause no side effects or locking conflicts.

3. READ → WRITE CONFLICT (UNREPEATABLE READ)

Occurs when transaction T1 reads item A, then T2 modifies and commits A, causing T1's second read to yield a different value.

Step 1 T1 Reads: T1: **READ(A)** gets A = 2 (deciding reservation).

Step 2 T2 Reads: T2: **READ(A)** gets A = 2 before T1 acts.

Step 3 T2 Writes & Commits: A = A - 2 = 0, T2: **WRITE(A)**, T2: **COMMIT**. Database now holds A = 0.

Step 4 T1 Reads Again: T1: **READ(A)** now gets A = 0 instead of 2.

Memory Trick — Unrepeatable Read

READ(A) → Interleaving WRITE(A) → READ(A) again → Different Value!

4. DETAILED EXAMPLE: LIBRARY MANAGEMENT SYSTEM

This scenario demonstrates how unrepeatable reads and interleaved execution lead to a **Lost Update** problem in practice.

Initial Database State: Book A = 10 copies

Time Step	Transaction 1 (T1)	Transaction 2 (T2)	DB Value
t1	READ(A) → 10	—	10
t2	Calculates $A = 10 - 1 = 9$	READ(A) → 10	10
t3	—	Calculates $A = 10 - 1 = 9$	10
t4	—	WRITE(A) → 9, COMMIT	9
t5	WRITE(A) → 9, COMMIT	—	9



Expected Logical Outcome

Initial Copies = 10
T1 issues 1 book + T2 issues 1 book
Expected Final Value = 8



Actual Database Outcome

T1 overwrote T2's commit
Final Database Value = 9
Result: 1 Update Lost!

5. CONCURRENCY PROBLEMS MASTER SUMMARY

Key mapping of operation combinations to their corresponding concurrency anomalies for rapid exam revision:

Combination	Short Code	Problem Name	Key Consequence
Read → Read	R - R	No Conflict	Safe parallel execution. No data modified.
Read → Write	R - W	Unrepeatable Read	Transaction gets different results for same read.
Write → Read	W - R	Dirty Read	Reading uncommitted data that may roll back.
Write → Write	W - W	Lost Update	Overwriting another transaction's uncommitted write.



Super Memory Trick & One-Line Revision

- **R - R:** No Problem
- **R - W:** Unrepeatable Read
- **W - R:** Dirty Read
- **W - W:** Lost Update



One-Line Summary: T1 READ → T2 WRITE → T1 READ again → Different Value = Unrepeatable Read.